A2 – S4 – 2025/2026 – II.2415
Advanced Algorithms and Programming
Semester Project

LAB_8

More Tree Structures

Session Objectives

By the end of this session, you will be able to:

1. Implement and use Binary Search Trees (BST) for user search and friend-of-friend suggestions
2. Use a Binary Heap for a trending posts feed in a social network
3. Build a Trie for username autocomplete
4. Build a Segment Tree for range queries on user activity
5. Compare & choose appropriate tree structures based on social network operations

Core Exercises

Exercise 1: Binary Search Trees – User Search & Friend-of-Friend Suggestions
Objective: Implement a BST to manage user profiles by user_id and efficiently find friends-of-friends for friend recommendations.

Social networks need fast lookup by user ID and the ability to explore friendship chains. A BST provides O(log n) average search/insert.

Requirements:

1. Implement UserBST structure - each node contains:
    - user_id (int, key)
    - name (string)
    - friends (list of user_ids)
    - left, right children
2. Implement core BST operations:
    - insert(user_id, name, friends_list)
    - find(user_id) → return user or None
    - inorder_traversal() → return sorted list of user_ids
    - delete(user_id)
3. Implement friend-of-friend (FoF) suggestions: suggest_friends(user_id, max_suggestions=5)
    - Find all friends of friends not already direct friends
    - Count frequency of each FoF
    - Return top max_suggestions by frequency
4. BST analytics:
    - get_height() → height of tree
    - is_balanced() → check if tree is balanced (height difference ≤ 1 at every node)
    - get_leaf_count() → number of leaf nodes

Complexity Analysis Questions:

1. What is the worst-case time complexity of find() in a BST? When does this happen in social networks?
2. If you insert users in increasing user_id order, what shape does the BST take? How does this affect suggest_friends()?
3. For a social network with 1 million users, what is the average height of a BST if insertions are random? Worst case?

Exercise 2: Binary Heap – Trending Posts Feed

Objective: Use a Max-Heap to maintain the top K trending posts by likes in real time.

Social networks need to quickly show the most popular posts. A binary heap gives O(log n) insert/update and O(k log n) for top k.

Requirements:

1. Implement TrendingHeap structure
    - Max-heap property (highest likes at root)
    - Store entries as tuples: (likes, post_id, timestamp)
2. Core heap operations:
    - push(post_id, likes, timestamp)
    - pop_max() → return post with most likes
    - peek_max() → view without removing
    - get_top_k(k) → return list of top k posts (without destroying heap)
    - update_likes(post_id, new_likes, timestamp) → update and reheapify
    - size() → number of posts in heap
3. Heap analytics:
    - is_valid_heap() → verify heap property
    - get_height()
    - get_level_order() → return list of heap level by level
4. Simulate trending feed:
    - Start with 100 posts (random likes 0–1000)
    - Perform 10,000 like updates
    - After every 1,000 updates, query and display top 5 posts
    - Measure average time per operation

Complexity Analysis Questions:

1. What is the time complexity of get_top_k(k)? Why is it better than sorting all posts?
2. If you used a sorted array instead of a heap, what would be the cost of update_likes()?
3. How would you modify the heap to automatically remove posts older than 24 hours?

Exercise 3: Prefix and Range Trees – Autocomplete & Activity Range Queries

Objective: Implement a Trie for username autocomplete and a Segment Tree for querying user activity over time.

Social networks need fast prefix search (search-as-you-type) and range queries (posts in last 7 days).

Part A: Trie for Autocomplete

Requirements:

1. Implement AutocompleteTrie structure - each node contains:
    - children[26] (or dict for lowercase letters)
    - is_end_of_username (bool)
    - user_id (if end of username)
2. Core Trie operations:
    - insert(username, user_id)
    - search(username) → return user_id or None
    - starts_with(prefix) → return True if any username starts with prefix
    - autocomplete(prefix, max_results=10) → return list of (username, user_id) matching prefix
3. Trie analytics:
    - count_words() → total number of usernames stored
    - get_height() → longest username length
    - get_total_nodes()
    - delete(username) → remove username if exists
4. Test with 50,000 usernames (realistic: alice, bob, alice123, etc.)

Complexity Questions:

1. Time complexity of autocomplete(prefix) in terms of prefix length and number of results?
2. Space complexity for 1 million usernames (average length 12)?
3. Why use a Trie over a hash map for autocomplete?

Part B: Segment Tree for Activity Range Queries

Requirements:

1. Implement ActivitySegmentTree structure
    - Stores daily post counts for n days (default n = 30 or 365)
    - Build tree from initial activity array
2. Core segment tree operations:
    - build(activity_array) → create segment tree
    - query(l, r) → return total posts between day l and day r (inclusive)
    - get_range_max(l, r) → return max posts in day range
    - get_range_min(l, r) → return min posts in day range
3. Segment tree analytics:
    - get_tree_size() → number of nodes in segment tree
    - get_height()
    - get_leaf_values() → return original activity array
4. Simulate social network activity:
    - Start with 30 days of random activity (0–1000 posts/day)
    - Query 7-day rolling totals for the last week

Complexity Questions:

1. Time complexity of query(l, r)? Prove it.
2. Space complexity of a segment tree for 365 days?
3. If you used a prefix sum array instead of a segment tree, what would be the cost of updates? When is each better?

Final Integration Questions

1. Representation Choice
   - For 1M users, would you use a BST or a hash map for user lookup? Justify with time and space trade-offs.
   - If you use a BST, what insertion order would cause worst-case performance? How would you fix it without rebalancing?
2. Structure Selection
   - To recommend friends-of-friends, why use BST traversal instead of BFS on a graph? What information does the BST provide that a graph doesn't?
   - For trending posts, why is a heap better than keeping a sorted list? What happens to performance if you need top 100 instead of top 10?
3. Scalability Considerations
   - If autocomplete must support 1 million queries per second, how would you optimize the Trie? (Hint: caching, compression)
   - For a Segment Tree with 365 days, what is the recursion depth? Is it safe? If not, how would you implement iteratively?
4. Performance Analysis
   - Calculate the time complexity of generating 5 friend suggestions for a user with 150 friends, where each friend also has 150 friends.
   - How many operations to query the last 7 days of activity using a Segment Tree? Using a prefix sum array? Which is faster if updates happen every second?

Edge Cases to Consider

1. What happens if you try to delete a username from the Trie that doesn't exist?
2. What happens if get_top_k(1000) is called but only 50 posts exist in the heap?
3. What happens if the BST becomes a degenerate chain (height = 1,000,000)? How would you detect and warn the user?
4. What happens if the Segment Tree receives an update with day = -1 or day > n-1?

General Instructions

Repository Setup
Create branch: LAB _08_MoreTrees

File Structure:
Follow the same structure of LAB _07

Git Requirements:
Same as for LAB _07

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 5	**Note: 15**
• Each Team (of number BB) Upload at the **end of the session** a very short PDF document and name it **TBB_LAB8_AES.pdf**, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name **TBB_LAB8_BFM.pdf** and upload it before **Friday midnight of the same week**, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.